INVESTIGACIÓN Y APLICACIÓN DEL PRINCIPIO FIFO

1. El Principio FIFO

El principio **FIFO** (acrónimo del inglés *First In, First Out*, traducido como «Primero en entrar, primero en salir») es un paradigma fundamental de organización y gestión de datos. Bajo este criterio, el primer elemento que se incorpora a una colección es estrictamente el primero en ser extraído o procesado, preservando de manera rigurosa el orden cronológico de llegada.

En las ciencias de la computación, este comportamiento se materializa a través del Tipo Abstracto de Datos (TAD) denominado Cola (Queue). A diferencia de otras estructuras lineales como los arreglos o listas enlazadas generales —donde se puede acceder, insertar o eliminar datos en cualquier posición arbitraria—, una cola restringe las operaciones a sus dos extremos:

- **Extremo posterior (Rear / Back):** Es el punto exclusivo de entrada. Toda nueva inserción (operación Enqueue) se realiza al final de la estructura.
- **Extremo frontal (Front / Head):** Es el punto exclusivo de salida. Toda extracción o consulta del siguiente elemento a procesar (operación Dequeue) se ejecuta desde el frente.

1.1. Aplicaciones

Las colas FIFO son indispensables en escenarios donde la equidad de atención, el almacenamiento temporal (buffering) y el procesamiento secuencial son críticos:

- **Sistemas Operativos:** Planificación de procesos (Round Robin y colas de listos), gestión de interrupciones y colas de impresión (Spooling).
- **Redes y Telecomunicaciones:** Buffers en routers y switches para encolar paquetes de datos antes de ser transmitidos según su orden de llegada.
- **Algoritmos de Grafos:** Recorrido en anchura (BFS - Breadth-First Search), esencial para encontrar el camino más corto en grafos no ponderados.
- **Sistemas Asíncronos y Microservicios:** Brókers de mensajería (como RabbitMQ o Apache Kafka) para desacoplar productores y consumidores de eventos.

2. Implementación en C++: La Clase std::queue

La Biblioteca Estándar de Plantillas (STL) de C++ proporciona una implementación robusta y altamente optimizada del principio FIFO mediante la plantilla **`std::queue`**, definida en el archivo de cabecera `<queue>`.

Técnicamente, std::queue es un adaptador de contenedor (container adapter). Esto significa que no gestiona la memoria directamente desde cero, sino que envuelve a otro contenedor secuencial subyacente (por defecto std::deque, aunque también admite std::list) y limita su interfaz pública únicamente a las operaciones válidas del modelo FIFO.

3. Ejemplo Práctico en C++: Sistema de Tickets

El siguiente código fuente implementa un simulador de atención de solicitudes de soporte técnico. Demuestra cómo se encolan estructuras personalizadas (struct Ticket), cómo se inspeccionan los extremos y cómo se procesan los registros respetando estrictamente el orden de llegada.

```
#include <iostream>
#include <queue>
#include <string>

// Estructura que representa cada solicitud ingresada al sistema
struct Ticket {
    int id;
    std::string cliente;
    std::string incidencia;
};

int main() {
    // Declaración de la cola FIFO utilizando la STL
    std::queue<Ticket> colaSoporte;

    // 1. Fase de inserción (Enqueue) mediante emplace/push
    std::cout << "=== RECEPCION DE TICKETS ===\n";
    colaSoporte.push({101, "Ana Gomez", "Error de autenticacion en portal"});
    colaSoporte.push({102, "Carlos Ruiz", "Solicitud de cambio de clave"});
    colaSoporte.push({103, "Elena Torres", "Discrepancia en facturacion"});

    std::cout << "Tickets en espera: " << colaSoporte.size() << "\n\n";

    // 2. Consulta de extremos sin alterar la estructura
    std::cout << "Proximo en ser atendido (front): #" << colaSoporte.front().id
              << " - " << colaSoporte.front().cliente << "\n";
    std::cout << "Ultimo en ingresar (back):       #" << colaSoporte.back().id
              << " - " << colaSoporte.back().cliente << "\n\n";

    // 3. Procesamiento secuencial bajo principio FIFO (Dequeue)
    std::cout << "=== PROCESAMIENTO DE ATENCION (FIFO) ===\n";
    while (!colaSoporte.empty()) {
        // Obtenemos referencia constante al elemento del frente
        const Ticket& actual = colaSoporte.front();

        std::cout << "Atendiendo Ticket #" << actual.id
                  << " | Cliente: " << actual.cliente
                  << " | Motivo: " << actual.incidencia << "\n";

        // Extraemos el elemento procesado de la cola
        colaSoporte.pop();
    }

    // 4. Verificación final de estado
    if (colaSoporte.empty()) {
        std::cout << "\n[OK] Todos los tickets han sido procesados en orden de
```

```
llegada.\n";
    }

    return 0;
}
```

3.1. Salida Esperada en Consola

```
=== RECEPCION DE TICKETS ===
Tickets en espera: 3

Proximo en ser atendido (front): #101 - Ana Gomez
Ultimo en ingresar (back):       #103 - Elena Torres

=== PROCESAMIENTO DE ATENCION (FIFO) ===
Atendiendo Ticket #101 | Cliente: Ana Gomez | Motivo: Error de autenticacion en portal
Atendiendo Ticket #102 | Cliente: Carlos Ruiz | Motivo: Solicitud de cambio de clave
Atendiendo Ticket #103 | Cliente: Elena Torres | Motivo: Discrepancia en facturacion

[OK] Todos los tickets han sido procesados en orden de llegada.
```